



Signal data Input / Output

信号データの入出力

SILAA, Victor Alex
silaa@kushiro.kosen-ac.jp

1. Be able configure structure data files .
2. Implement input / output processing for the WAV format (recording method for digital data of audio signals)

a. 音声信号のデジタルデータの記録方法 (WAV形式)

method of recording digital data of audio signal (WAV format)

b. 入出力処理の実装方法を理解

Understand how to implement input / output processing

1. WAV ファイルの調査・入手

Search and obtain WAV file

2. WAV 形式の構成

Structure of WAV format

3. WAV ファイルの入出力処理

WAV file input / output processing



既存の WAV ファイル(ファイル.wav)を探索.

1. Search for existing WAV file. Ex. → (file.wav)

- \$ **ls -R /usr/share/sounds/ | less**
- # とか (And)
- \$ **find /usr/share/sounds/ -name *.wav**

アプリの効果音等が他のディレクトリにもあるかも. ネット上のフリー素材をダウンロードしてもOK!

2. sound effects of the app may also be found in other directories. U can download free materials on the internet.

音声を再生しつつ, PCM 形式の種類も確認して, 実験対象とする WAV ファイルを選択せよ.

3. While playing the audio, check the PCM format type and select the WAV file for the experiment.

- `$ paplay -v ディレクトリ/ファイル.wav`

サウンドサーバ PulseAudio 用の音声再生コマンドです. とりあえず「サンプル仕様 's16le ...」(整数16bitリトルエンディアン)なものの限定で選んでおこう.

This is an audio playback command for sound server PulseAudio. For now, just limit selection to "sample specification's 16le" if u want to playback(no need to check format) omit -V

再生したいだけ(形式確認が不要)なら, 引数の「-v」は省略OK

音声の波形を観察せよ

4. Observe the audio waveform.

- `$ audacity ディレクトリ/ファイル.wav &`

音声エディタ Audacity です.とりあえず, 波形表示機能だけを利用させていただきましょう. 操作方法については, 各自で調査しよう →

It is audio editor Audacity. For now, lets just use the waveform display function. Do your own research on how to use it.

Install → proxy settings = export http_proxy=pxxxxx:pwd

WAV 形式の仕様について，教科書を参照せよ

1. Please refer to your textbook for the WAV format specifications.

```
$ hexdump -C ディレクトリ/ファイル.wav | less
```

WAV ファイル内のバイナリデータを観察せよ

2. Observe the binary data in the WAV file.

hexdump は，人間には読みづらいバイナリデータを可読な形式（16進数字列や英字）に変換するコマンドです．WAV は，テキストファイルではないので，less 単独では読めませんよ．

Hexdump command converts binary data that is difficult for humans to read into a readable format (hexadecimal or alphanumeric). WAV is not a text file, so it cannot be read by less alone.

サンプルプログラム

sample program

作業用ディレクトリの準備

3. Preparing working directory.

```
$ mkdir sp    # ディレクトリ名は任意で OK directory name  
is optional  
  
$ cd !$
```

ソースディレクトリ: [pcm/](#) (個別ダウンロード)

4. Source directory : PCM/(separate download) →

圧縮ソースファイル: [pcm.tgz](#) (一括ダウンロード)

5. Compressed source file : PCM.tgz (bulk download)



展開

6. Deployment.

```
$ tar zxvf pcm.tgz
```

```
pcm/
```

```
...
```

コンパイル

7. Compile

```
$ cd pcm/
```

```
$ make
```

```
...
```

```
$ ls
```

```
... libpcm.so      rate  sin    thru  ...
```

WAVファイル入出力ライブラリとサンプルアプリ3個です

→ WAV file input / output library and 3 sample applications.



動作テスト `$ ./sin | paplay` # 鳴りますか? Does it ring

Operation Test.

アプリの実行に失敗した場合, まずは環境設定を見直そう.

If the app fails to run, first check the environment settings.

```
$ vim ~/.bashrc  
  
...  
  
export LD_LIBRARY_PATH=.  
  
$ source ~/.bashrc
```

「図形処理」で設定したはず

It should have been set in graphic processing



ドキュメントとコードを確認し，さらにテストしよう

Check documentation and code and test it further.

```
$ less 00ReadMe.txt
...
$ view sin.c
...
$ view pcm.h
...
$ view pcm.c
...
```

view は(多分)Vim エディタの表示専用モードです。つまり，vim とか他のエディタで開いてもOKですが，不用意に内容を書き換えないように注意。

View is the Vim editor's display-only mode. In other words, you can open it with vim or another editor, but be careful not to change the contents carelessly.



C言語によってバイナリデータを効率的に処理するには、標準ライブラリ関数 `fread()`, `fwrite()` と `memcpy()` の利用が鍵となります。

The key to efficiently processing binary data using C language is to use standard library functions `fread()`, `fwrite()` and `memcpy()`.

`fread()`: ファイルからメモリ領域へバイナリデータを入力

`fread()` : input binary data from memory area to file

fwrite(): メモリ領域からファイルへバイナリデータを出力

fwrite() : Output binary data from memory area to file

memcpy(): メモリ領域から別のメモリ領域へバイナリデータをコピーする

memcpy() : Copy binary data from one memory area to another.

今後の課題に備え, マニュアル等を調べておこう. ライブラリのソース pcm.c 内でも使用しています

In preparations for future issues, lets look into manuals etc. It is also used in the library source pcm.c

1. サンプルプログラムを利用・改造し，WAV ファイル入出力処理の実装方法を理解せよ

Use and modify the sample program to understand how to implement WAV file input/output processing

2. アプリのソース `sin.c` を改造し，PCM データ属性（量子化深度，チャンネル数，標本化周波数）の異なる 複数の WAV ファイルを作成せよ．そして，それらのファイルのヘッダ部（“RIFF”～“data”の間の部分，fmtチャンク）にある バイナリデータの差異を hexdump によって分析せよ．

Modify the application source **sin.c** and create multiple WAV files with different PCM data attributes (**quantization depth, number of channels, sampling frequency**). Then use **Hexdump** to analyze the differences in binary data in the header section (the part between “**RIFF**” and “**data**” , fmt chunk) of those files. Example:



```
$ ./sin A 8 > a-8-1-48.wav# 8bit, 1ch, 48kHz, 無改造で OK
                                # unmodified
$ ./sin A 16 > a-16-1-48.wav      # 16it, ...
...
$ vim sin.c# fs = 0xFF12;        とか分かりやすい数字に改造
                                # とりあえず, 標本化定理なんて無視して, テキトーで OK
                                #modify to a number that is easy to understand. For now ignore the
sampling theorem and just go with what u know
$ make
$ ./sin A 8 > a-8-1-FF12.wav
$ ...
$ hexdump -C a-○-○-○.wav | less
...
```

1. 各バイナリデータは WAV 形式の仕様の通りでしょうか？

Is each binary data as per WAV format specification?

2. チャンネル数と標本化周波数の変更には, ソースコードの書き換えが必要

Changing the number of channels and sampling frequency requires rewriting the source code.



1. アプリのソース `sin.c` および `thru.c` を手がかりとして、ライブラリのソース `pcm.c` 内の関数 `pcmInit()`, `pcmRead()`, `pcmWrite()`, 等の処理内容を解剖せよ

Using the application source `sin.c` and `thru.c` as clues, dissect the processing of the functions `pcmInit()`, `pcmRead()`, `pcmWrite()`, etc. in the library source `pcm.c`

2. サンプルプログラムを利用し、再生速度の変更について理解せよ

Using the sample program, understand how to change the playback speed.

→ アプリ `rate` を利用し、標本化周波数を変更して音声を再生し、効果を聴覚および `audacity` によって観察せよ

Use the app `rate` to play back audio by changing the sampling frequency, and observe the effect by hearing and `audacity`.



デジタルらしく、速度だけ変わって欲しいところですが、
ピッチ（音程）も変わってしまい、アナログ感... 速度だけ
変更には、それなりの処理が必要ということね

I would like only the speed to change, like digital, but the pitch also changes, giving it an analog feel. I guess that only changing the speed requires a certain amount of processing.

次回予告

Preview of the next installment

1. 複数の音の WAV ファイルを連結し，メロディを奏でよう

Lets play a melody by concatenating WAV files of multiple sounds.

2. 周波数の異なる正弦波を重ね合わせたり，振幅を変調して，現実の楽器のような音色を生成しよう

Generate realistic instrument-like tones by superimposing sine waves of different frequencies and modulating their amplitude.



Thank you for listening !